

PORTFOLIO PROJECT // 2026.10

NULL POINT

익스트랙션 슈터의 그리드 인벤토리와 레이드 루프

델타 포스를 레퍼런스로 , 로비 · 인벤토리 · 탈출 루프를 UE5 블루프린트로 구현한 개인 프로젝트

장비 칸 배치 72 종

아이템 514 종

탈출 카운트다운

[이름] · [과정 / 기수]

Unreal Engine 5.8 · Blueprint | 2026.09.10 ~ 10.05



Contents

00

프로젝트 개요

01

게임 루프 · 탈출

02

핵심 기술 · 그리드 인벤토리

03

성능 최적화

04

트러블 슈팅 · 구조 개선

05

성장 · 향후 과제

프로젝트 개요



한 줄 설명

로비에서 장비를 꾸리고 → 작전 지역에서 탈출 구역 10 초를 버티면 → 로비로 복귀

핵심 : 아이템 · 장비마다 칸이 다른 그리드 인벤토리를 데이터 기반으로 구현

기본 정보

기간 2026.09.10 ~ 10.05 (약 4 주) · 개인
엔진 Unreal Engine 5.8 · Blueprint
장르 익스트랙션 슈터 (델타 포스 레퍼런스)

72

칸 배치 데이터 (장비 · 상자)

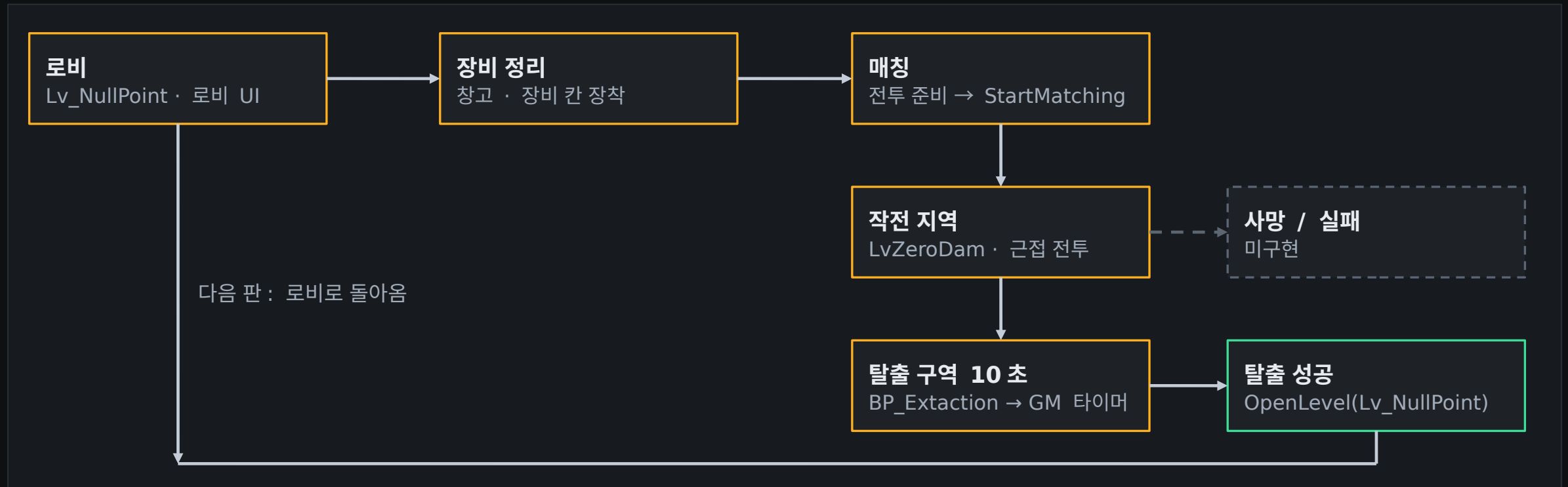
514

아이템 정의

29

직접 만든 블루프린트

게임 흐름

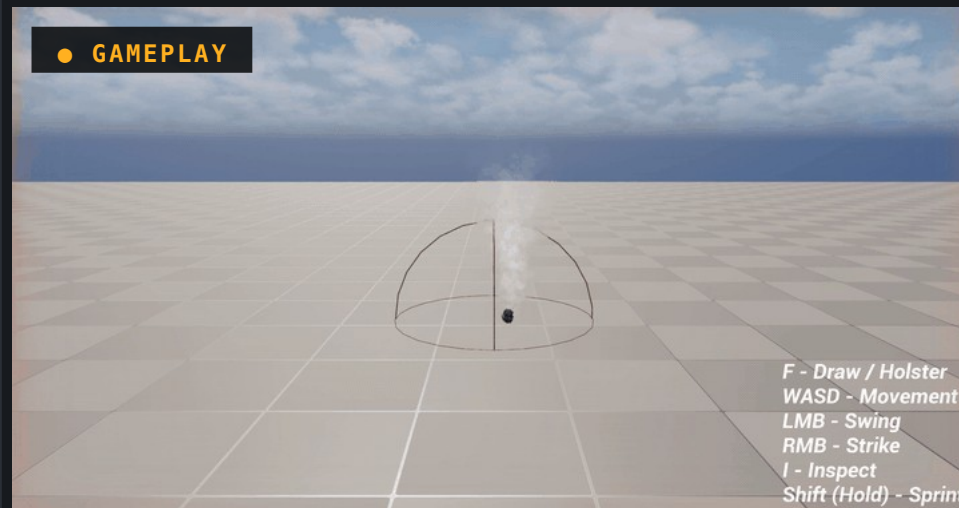


시스템 개입 지점 ① 매칭 — BP_NullPointGM ② 탈출 판정 — BP_Extraction → GameMode 타이머 ③ 레벨 전환 — OpenLevel

탈출 루프 · 10 초 카운트다운

```
// BP_Extaction - 탈출 구역
on BeginOverlap(Player) → GM.StartExtractUser() // 탈출 시작
on EndOverlap(Player) → GM.StopExtractUser() // Time = 100

// BP_FirstPersonGameMode - 0.1 초 반복 타이머
Timer():
    if not bIsExtracting: return
    Time -= 1 // 100 → 0 = 10 초
    if Time % 10 == 0: HUD.PlayCountDownWidget(Time / 10)
    if Time == 0: PauseTimer(); OpenLevel("Lv_NullPoint")
```



실제 플레이 · 구역에 머물면 카운트다운 → 0 이면 로비 복귀

01

구역은 판정만

규칙은 GameMode 한 곳 — 구역을 여러 개 놓아도 그대로

02

오차 없는 10 초

0.1 초 타이머 × 정수 카운터 (float 누적 없음)

03

중복 호출 방지

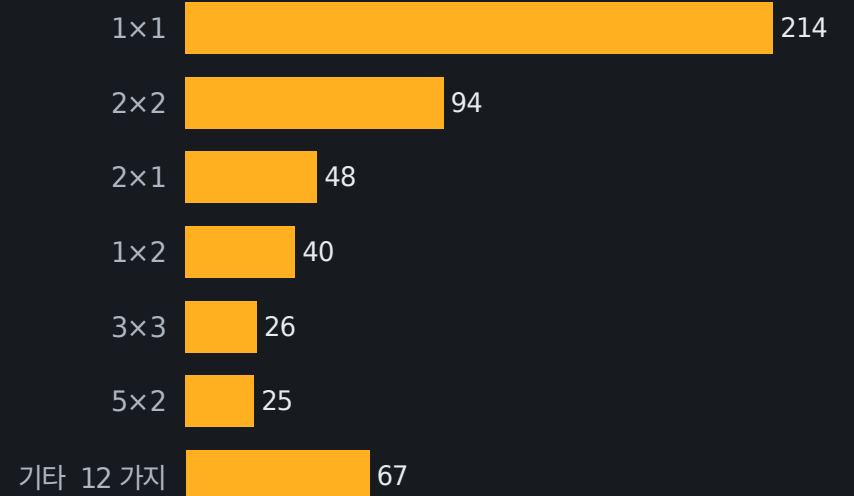
0 이 되면 타이머부터 멈추고 레벨 전환

그리드 인벤토리 · 칸 · 아이템 · 섹션



칸 번호 = $X + Y \times \text{가로 칸 수}$ 예) 헬멧의 왼쪽 위 칸 = $2 + 0 \times 8 = 2$

아이템 514 종 · 칸 크기 분포



칸 · WBP_GridCell

64px 한 칸. 왼쪽 위부터 0, 1, 2... 번호를 매기고, 아이템이 놓였는지 (bIsPlacedItem) 를 가진다.

아이템 · 가로 × 세로 칸

DT_ItemDefinition 의 GridWidth × GridHeight. 왼쪽 위 칸부터 W × H 칸을 차지한다.

섹션 · WBP_GridSlot

칸 묶음 하나 = 작은 그리드 하나. 창고는 9x25 섹션 1 개, 주머니는 1x1 섹션 5 개, DARAssault 조끼는 섹션 7 개.

데이터 구조 · DataTable 중심 설계

위젯마다 하드코딩하지 않고, 데이터 (DataTable · DataAsset) 와 화면 (Widget) 을 나눴다

데이터	담는 것
DT_GridLayoutData · 72 행	조끼 18 · 가방 26 · 창고 · 상자의 칸 배치 (섹션 크기 · 위치 · 반 칸)
DT_ItemDefinition · 514 행	아이템 이름 · 등급 · 가로 / 세로 칸 · 아이콘
ST_StashItemInfo	창고에 놓인 아이템 (슬롯 · 칸 번호 · 아이템) — 시작 아이템 9 개 배치
DA_UISetting	칸 크기 (64) 등 UI 공통 값을 한 곳에서 관리

이름으로 연결된 데이터

장착하면 이 이름으로 DrawLayout → 칸 배치

아이템 이름 · DT_ItemDefinition

Gear_ChestRig_DARAssault

=

칸 배치 행 이름 · DT_GridLayoutData

Gear_ChestRig_DARAssault

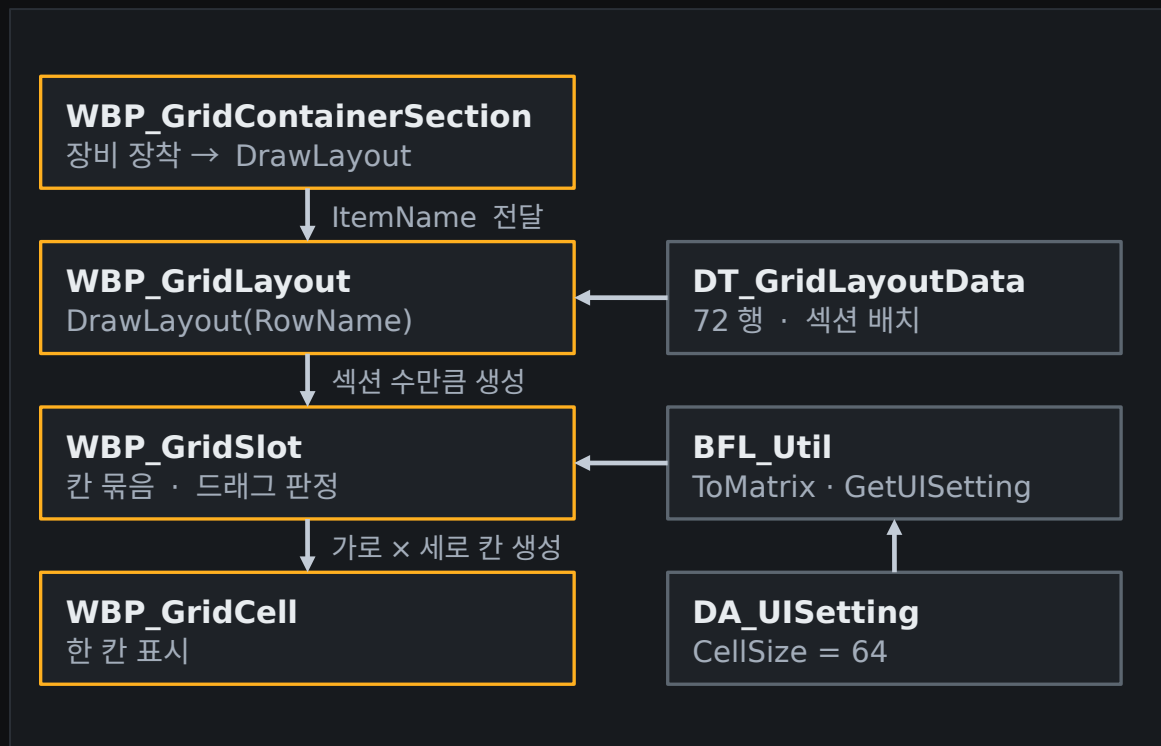
칸 배치 · 구조와 역할

시스템 개요

장비를 장착하면 GridLayout 이 행 이름을 받아 섹션마다 GridSlot 을 만들고, 슬롯이 칸 (GridCell) 을 채운다.

구현 의도

반 칸씩 어긋난 조끼 칸 → GridPanel 2 배 해상도 + ToMatrix 좌표 변환.
72 종을 손으로 짜지 않기 위한 선택.



DT 행 → 실제 칸 배치

체스트 리그



0	1	0	1
0	1	0	1
2	3	2	3
0	0	0	1
1	1	2	3
2	2	4	5

DARAssault · 섹션 7 개
실제 게임 화면



DRCAdvancedRecon · 반 칸
DT 데이터로 재구성

칸 배치 · 핵심 구현

```

DrawLayout(RowName)    // 예 : "Gear_ChestRig_DARAssault"
    Row = GetDataTableRow(DT_GridLayoutData, RowName)
    for Section in Row.LayoutSections:
        Slot = CreateWidget(WBP_GridSlot, Section.SectionSize)
        Pos  = ToMatrix(Section.GridPosition, bHalfRow, bHalfCol)
        Span = ToMatrix(Section.SectionSize)
        S = GridPanel.AddChildToGrid(Slot, Pos.Row, Pos.Column)
        S.SetRowSpan(Span.Row); S.SetColumnSpan(Span.Column)

ToMatrix(P, HalfRow, HalfCol)    // BFL_Util
    return Row = P.Y*2 + HalfRow, Column = P.X*2 + HalfCol

```

핵심

입력 DT 행 이름 하나 (장착한 장비의 ItemName)

처리 섹션마다 슬롯 생성 → ToMatrix 2 배 좌표 → GridPanel
에 배치

결과 72 종 모두 같은 함수 하나로 그려진다

ToMatrix 한 줄 (×2, 반 칸이면 +1) 이 반 칸 배치의 전부

DT 1 행 새 장비 추가 비용

1 곳 칸 크기 설정 ·
DA_UISetting

3 단계 Layout · Slot · Cell 역할
분리

드래그 앤 드롭 · 사용 흐름

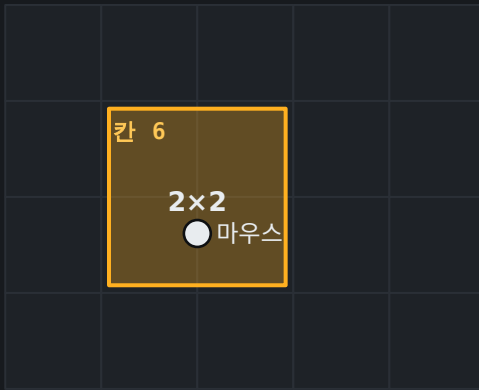


실제 플레이 · 창고 안에서 헬멧 (H70Elite) 을 다른 칸으로 옮기기

- 1 누르기
OnMouseButtonDown → DetectDragIfPressed
- 2 드래그 시작
BP_DragDrop 생성 (아이템 정보 · 원래 위젯)
- 3 그리드 위 이동
OnDragOver: 칸 번호 계산 → CanPlaceIndex 기록
- 4 놓기
OnDrop: 그 칸에 배치 / 장비 칸이면 칸 배치 교체

드래그 앤 드롭 · 배치 판정

① 마우스 → 칸 번호



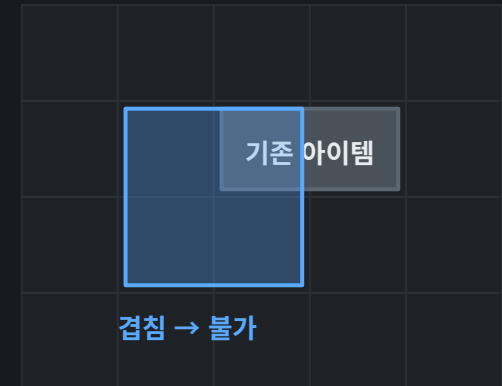
아이템 중심을 마우스에 맞추고, 왼쪽 위 칸 = $(\text{마우스} - \text{크기} \times 32\text{px}) \div 64\text{px}$

② 범위 클램프



그리드 밖으로 나가면 가장자리 칸에 붙인다 — $0 \leq \text{위치} \leq (\text{그리드} - \text{크기}) \times 64$

③ 겹침 검사



아이템이 덮는 칸 중 하나라도 이미 차 있으면 (blsPlaceItem) 놓을 수 없다

④ 놓기

행 = 번호 ÷ 가로, 열 = 번호 % 가로 → **AddChildToGrid + RowSpan(H) · ColumnSpan(W)**

개발 중: 회전 (가로 · 세로 교체, GetEffectiveItemSize) 과 짝수 칸 중심 보정 (SnapDragAxis) 은 함수까지 구현했고 판정 연결 전. 놓은 칸의 점유 기록 (blsPlaceItem 쓰기) 도 남은 과제.

드래그 앤 드롭 · 핵심 구현

A · 판정 — 마우스 좌표를 칸으로

```
OnDragOver(Geometry, PointerEvent, Op) // GridSlot
    Mouse = AbsoluteToLocal(Geometry, PointerEvent)
    Size = Op.ItemInstance.ItemDef.(Width, Height)
    Cell = GetUISetting().CellSize // 64
    // 아이템 중심을 마우스에, 그리드 밖으로는 못 나가게
    Pos = Clamp(Mouse - Size*Cell/2,
                0, (Grid - Size)*Cell)
    Index = Pos.X/Cell + (Pos.Y/Cell) * Grid.X
    Op.bCanPlaceItem = Func_CanPlaceItem(Index, Size)
    if Op.bCanPlaceItem: Op.CanPlaceIndex = Index
```

B · 실행 — 판정 결과대로 배치

```
OnDrop(Geometry, PointerEvent, Op) // GridSlot
    if not Op.bCanPlaceItem: return
    Item = CreateWidget(WBP_ItemSlot, Op.ItemInstance)
    S = GridPanel.AddChildToGrid(Item,
        Row = Op.CanPlaceIndex / Grid.X,
        Column = Op.CanPlaceIndex % Grid.X)
    S.SetRowSpan(Height); S.SetColumnSpan(Width)
    Op.ItemSlot.RemoveFromParent() // 원래 자리 제거
```

핵심

A 는 판정만 , B 는 배치만 한다 . 그 사이 데이터는 BP_DragDrop 이 나르기 때문에 , 그리드는 드래그를 시작한 위젯이 무엇인지 몰라도 된다 .

남은 과제 : 놓은 칸의 점유 기록 · 회전 연결

장비 장착 → 칸 배치 교체



① 체스트 리그를 장비 칸에 놓으면 → 조끼 칸 배치가 그려짐



② 가방을 장비 칸에 놓으면 → 가방 칸 배치가 그려짐

구현은 한 줄

OnDrop → `EquipmentSlot.SetItem()` → `GridLayout.DrawLayout(ItemDef.ItemName)`

아이템 이름 = 칸 배치 행 이름이라, 조끼 18 종 · 가방 26 종이 같은 코드로 동작한다.

성능 최적화 · 그리드 위젯 수

병목

1,350

창고 225 칸 × 칸당 위젯 6 개

창고 그리드를 칸마다 위젯 (WBP_GridCell) 으로 만든다 . 가장 큰 창고 (9×51) 는 2,754 개 .

원인

× 6

칸 하나당 UMG 위젯

칸 대부분은 움직이지 않는 격자선인데 위젯으로 남아 , Slate 가 위젯 수만큼 레이아웃 · 페인트를 처리한다 .

개선 계획

225 → 0

셀 위젯 수 (구조상 계산)

격자선은 Image 한 장이나 OnPaint 한 번으로 그리고 , 점유 여부는 bool 배열로 관리한다 .

측정 계획 : Unreal Insights · Widget Reflector 로 같은 조건 (창고 탭 첫 표시) 에서 개선 전후 비교

좌표 버그 · 문제와 원인

문제 현상

드래그한 아이템의 놓일 칸 표시가 안 나오고, 칸 계산이 화면과 맞지 않았다 (9/29, 커밋 5c83059).

확인 방법

계산식을 고치기 전에, 칸마다 번호를 띄우는 디버그 그리드부터 만들었다 (오른쪽 화면).

원인 3 가지

- ① BP_DragDrop 에 아이템 정보가 실리지 않아 크기가 0x0
- ② 셀 크기 변수가 둘 — 위젯은 64, 좌표 계산은 50
- ③ GridPanel 용 2 배 좌표를 1 배 셀 배열에 그대로 사용



칸 번호를 띄워 둔 실제 참고 화면

좌표 버그 · 해결 과정

수정 전 · 5c83059 (9/29 01:20)

```
CreateDragDropOperation(BP_DragDrop, Visual)
// 아이템 정보 없음
Size = Op.NewVar.(W, H) * 2 // → 0 × 0
Start = ((Mouse - (W-1)*50) / (2*50)) * 2 + ...
Cells[Getincludeindex(...)].SetColor() // 셀은 64px
```

수정 후 · ab3fcc0 → c708cb0 (9/29 ~ 9/30)

```
CreateDragDropOperation(BP_DragDrop, Visual,
    ItemInstance, NewVar = ItemDef, ItemSlot = self)
Cell = GetUISetting().CellSize // 64, 한 곳
Pos = Clamp(Mouse - Size*Cell/2, 0, (Grid-Size)*Cell)
Index = Pos.X/Cell + (Pos.Y/Cell) * Grid.X // 1배
```

바꾼 것

① 드래그 시작 시 아이템 정보 · 원래 슬롯을 함께 전달

② 셀 크기는 GetUISetting() 하나로만 읽기

③ 범위 제한 후 셀 크기로 나눠 1 배 칸 번호 계산

검증

디버그 칸 번호 = 계산된 칸 번호인지 확인 → 그 칸에 크기대로 놓이는지 확인. 오른쪽: 고친 뒤 헬멧이 제자리에 놓인 화면.

81	82	83	84	85	86	87	88	89
	92	93	94	95	96	97	98	
	101	102	103	104	105	106	107	
108	109	110	111	112	113	114	115	116

아이템 데이터 구조 개선

목표 : 아이템 514 종을 넣기 전에 , 공통 정의와 타입별 데이터 · 배치 상태를 나눈다

1 차 구조 · ~9/30

- 공통 구조체 하나 (ST_ItemDefinitionRow) 에 모든 필드
- 방어구 · 무기 고유 데이터를 둘 자리가 없음
- 구조체가 폴더마다 흩어짐



2 차 구조 · 10/1 (1c1cc33)

- ① **정의** ST_ItemDefRow · DT 514 행
- ② **확장** 타입별 ExtraDef · E_ExtraDataType 로 선택
- ③ **배치** ST_StashItemInfo · 슬롯 · 칸 번호

결과 아이템 추가 = DataTable 행 추가 · 커밋 한 번에 파일 32 개 정리 (타입별 확장 필드는 골격 단계)

성장한 점과 향후 과제

아쉬운 점

- 배치 결과를 위젯이 갖고 있어 저장 · 불러오기가 어렵다
- 회전 배치를 판정에 연결하지 못했다
- 레이드 상태가 bool 두 개로 흩어져 있다

향후 과제

- 인벤토리 컴포넌트가 배치 · 점유를 소유
- 회전 · 창고 저장 완성, E_RaidState 도입
- 판정 로직 C++ 이전 + 자동 테스트

배운 점

- 좌표 버그는 계산식보다 디버그 시각화부터
- 데이터와 표시는 처음부터 분리한다

발표를 들어주셔서 감사합니다 .

GitHub github.com/immigration2000/ExtractionGame_Fin · 시연 영상 [링크]